Misión de investigación: SQL + Python + IA

Emma Choque Sandoval

Durante esta actividad investigué cómo se conectan SQL y Python usando una IA como tutora. Para las pruebas usé una base de datos SQLite de práctica con cinco películas ficticias, sin descargar la de SQL Case Files. Todo el código lo ejecuté de verdad y los resultados que aparecen son los que me salieron.

Reto 1. Comprende los conceptos

Tu tarea: escribe las respuestas con tus propias palabras y un ejemplo de aplicación práctica.

1. ¿Qué es SQL?

SQL es el lenguaje que uso para pedirle cosas a una base de datos: ver datos, filtrarlos, contarlos, agregarlos o borrarlos. No es un lenguaje para hacer programas completos, sino para hablar con los datos. Ejemplo práctico: SELECT titulo FROM peliculas WHERE anio > 2018; me da los títulos de las películas posteriores a 2018.

2. ¿Qué es Python?

Python es un lenguaje de programación de propósito general. Sirve para hacer programas: calcular, repetir tareas, leer archivos, mostrar resultados, etc. Ejemplo práctico: un programa que calcula la media de unas notas y la muestra por pantalla.

3. ¿Cuál es la diferencia entre una base de datos y un lenguaje de programación?

La base de datos es donde están guardados los datos, ordenados en tablas. Un lenguaje de programación es la herramienta con la que escribo instrucciones para que el ordenador haga cosas. Una es el almacén y el otro es quien trabaja con lo que hay dentro. Ejemplo práctico: la tabla peliculas es la base de datos; el programa en Python que la consulta y muestra los resultados es el lenguaje de programación.

4. ¿Para qué sirve SQLite?

SQLite es un sistema de base de datos pequeño y sencillo que no necesita instalar ningún servidor: toda la base de datos puede estar en un solo archivo o incluso en la memoria. Es ideal para practicar, para apps pequeñas y para pruebas. Ejemplo práctico: la app de notas de un móvil guarda sus datos en un archivo SQLite.

5. ¿Por qué utilizaríamos SQL y Python juntos en un proyecto de análisis de datos?

Porque cada uno hace lo que mejor sabe. SQL saca y resume los datos de

la base de datos de forma rápida, y Python los procesa, calcula estadísticas, hace gráficos o los muestra. Juntos me permiten automatizar el análisis. Ejemplo práctico: sacar con SQL los intentos de login fallidos de una empresa y, con Python, calcular cuántos hay por hora y avisar si pasan de un límite.

Reto 2. Investiga cómo se conectan

Tu tarea: explica qué hace cada instrucción y cómo se relacionan entre sí.

- **import sqlite3:** carga el módulo que me permite trabajar con SQLite desde Python. Sin esta línea, Python no conoce los comandos de sqlite3.
- **sqlite3.connect():** abre la conexión con la base de datos. Si le paso un nombre de archivo, abre o crea ese archivo; si le paso ":memory:", crea la base en la memoria. Devuelve un objeto conexión.
- **execute():** ejecuta una instrucción SQL (un CREATE, un INSERT, un SELECT ...). Se usa a través del cursor, que sale de la conexión con conexion.cursor().
- **fetchall():** después de un SELECT, recupera todas las filas del resultado en una lista de tuplas. Existe también fetchone() para recuperar solo una fila.
- **close():** cierra la conexión cuando termino, para liberar la base de datos.

Cómo se relacionan: primero importo el módulo, después me conecto con connect(), con la conexión creo un cursor, con el cursor ejecuto la consulta con execute(), recupero los resultados con fetchall() y por último cierro con close(). El orden importa: si cierro antes de recuperar los resultados, da error (lo comprobé en el reto avanzado B).

Reto 3. Conviértete en detective de código

Pide a la IA un ejemplo mínimo, comentado y apto para principiantes, y responde las preguntas.

El ejemplo que usé es este (archivo ejemplo_basico.py). Lo ejecuté con una base de datos de prueba en memoria:

```
import sqlite3

conexion = sqlite3.connect(":memory:")
cursor = conexion.cursor()

cursor.execute("""
    CREATE TABLE peliculas (
        id INTEGER PRIMARY KEY,
        titulo TEXT,
```

```
        genero TEXT,
        anio INTEGER,
        puntuacion REAL
    )
""")
cursor.execute("INSERT INTO peliculas VALUES (1, 'El faro de cristal',
'Drama', 2018, 8.1)")
cursor.execute("INSERT INTO peliculas VALUES (2, 'Códigos en la
niebla', 'Misterio', 2021, 7.4)")
cursor.execute("INSERT INTO peliculas VALUES (3, 'La última estación',
'Drama', 2015, 7.9)")
cursor.execute("INSERT INTO peliculas VALUES (4, 'Risas en Barcelona',
'Comedia', 2019, 6.8)")
cursor.execute("INSERT INTO peliculas VALUES (5, 'Mar de silicio',
'Ciencia ficción', 2023, 8.5)")
conexion.commit()

consulta = "SELECT titulo, anio FROM peliculas WHERE genero = 'Drama'
ORDER BY anio"
cursor.execute(consulta)
resultados = cursor.fetchall()

for fila in resultados:
    print(fila)

conexion.close()
```

Resultado al ejecutarlo:

```
('La última estación', 2015)
('El faro de cristal', 2018)
```

1. ¿Dónde está escrita la consulta SQL?

En la variable consulta, dentro de un texto entre comillas: "SELECT titulo, anio FROM peliculas WHERE genero = 'Drama' ORDER BY anio". Para Python es solo un texto; el que lo entiende es SQLite.

2. ¿Qué información intenta recuperar?

El título y el año de las películas de género Drama, ordenadas por año de la más antigua a la más nueva.

3. ¿Qué parte del código ejecuta la consulta?

La línea cursor.execute(consulta). Antes de esa línea la consulta solo está escrita, no pasa nada.

4. ¿Cómo se reciben los resultados?

Con cursor.fetchall(), que devuelve una lista de tuplas (una tupla por fila) y la guardo en la variable resultados. Después recorro la lista con un for y la muestro con print.

5. ¿Qué cambiarías para recuperar otra información?

Cambiaría el texto de la consulta. Por ejemplo, para ver los títulos con puntuación mayor a 8: "SELECT titulo FROM peliculas WHERE puntuacion > 8". El resto del código queda igual.

Reto 4. Comprueba lo que has aprendido

La IA me propuso tres preguntas sin darme las soluciones hasta que respondiera. Estas fueron mis respuestas y la comprobación posterior.

Pregunta 1. ¿Qué diferencia hay entre execute() y fetchall()?

Mi respuesta: execute() ejecuta la consulta y fetchall() recupera los resultados. Revisión: correcta. execute() solo manda la consulta a la base de datos; sin fetchall() no tengo las filas en una variable.

Pregunta 2. ¿Qué pasa si hago close() antes de fetchall()?

Mi respuesta: da un error porque ya no hay conexión. Revisión: correcta. Lo comprobé en el reto B y salió sqlite3.ProgrammingError: Cannot operate on a closed database.

Pregunta 3. Escribe una consulta SQL para ver los títulos de las películas con puntuación mayor a 8.

SELECT titulo FROM peliculas WHERE puntuacion > 8;

Revisión: correcta. Devuelve El faro de cristal (8.1) y Mar de silicio (8.5).

Qué necesito repasar:

commit(): sé que sirve para guardar los cambios cuando hago INSERT, pero todavía me cuesta decidir cuándo es imprescindible. También quiero practicar fetchone() frente a fetchall().

Reto 5. Comparte tu descubrimiento

Publicación para el espacio compartido de la clase:

Una cosa nueva que he aprendido

Que el orden importa: hay que hacer fetchall() antes de close(). Si cierro la conexión antes, Python da error aunque la consulta esté bien.

Un ejemplo o fragmento de código que ahora comprendo

```
cursor.execute("SELECT titulo, anio FROM peliculas ORDER BY anio")
resultados = cursor.fetchall()
conexion.close()

for fila in resultados:
    print(fila)
```

Una duda que todavía tengo

¿Cuándo conviene usar fetchone() en vez de fetchall()? Sé que fetchone() trae una sola fila, pero no tengo claro si es mejor cuando hay muchísimos datos.

Una respuesta útil que he encontrado a la pregunta de otra compañera

Si te sale "no such table", revisá que el nombre de la tabla esté escrito igual en el CREATE TABLE y en el SELECT (a mí me pasó con pelicula y peliculas). Y si usás ":memory:", acordate de que la base solo existe mientras el programa corre: si cierras y volvés a abrir, la tabla ya no está.

Reglas para trabajar con IA

- No copié ninguna respuesta sin comprenderla.
- Pedí explicaciones más sencillas cuando las necesité.
- Comprobé el código ejecutándolo antes de darlo por correcto.
- Cuando algo falló, miré el mensaje de error para entenderlo.
- No compartí contraseñas, datos personales ni información privada.

Retos avanzados

Reto avanzado A. Crea tu propia base de datos

Creé una base SQLite con la tabla peliculas y cinco registros ficticios, ejecuté una consulta SQL y mostré los resultados desde Python. El código es el mismo del Reto 3 (ejemplo_basico.py): CREATE TABLE, cinco INSERT, commit() y una consulta SELECT. El resultado fue el que aparece arriba: dos películas de Drama ordenadas por año.

Reto avanzado B. Encuentra el error

Programa con dos errores (reto_b_con_errores.py):
cursor.execute("SELECT titulo, anio FROM pelicula ORDER BY anio")
conexion.close()
resultados = cursor.fetchall()
Error 1: la tabla se llama peliculas (en plural) y en el SELECT puse pelicula. Mensaje: sqlite3.OperationalError: no such table: pelicula. Lo reconocí porque el mensaje dice que no existe esa tabla y la que creé tiene otro nombre.
Error 2: hace close() antes de fetchall(). Con el primer error arreglado, el mensaje fue sqlite3.ProgrammingError: Cannot operate on a closed database. La solución es recuperar los resultados primero y cerrar después.
Versión corregida (reto_b_corregido.py):
cursor.execute("SELECT titulo, anio FROM peliculas ORDER BY anio")
resultados = cursor.fetchall()
conexion.close()
Resultado: ('El faro de cristal', 2018) y ('Mar de silicio', 2023). Aprendí que el segundo error no aparece hasta que arreglo el primero, porque el programa se detiene en la primera línea que falla.

Reto avanzado C. Mini análisis de datos

Recuperé las puntuaciones de la tabla, calculé la media con Python y la comparé con la que calcula SQL con AVG() (reto_c_mini_analisis.py):
cursor.execute("SELECT puntuacion FROM peliculas")
filas = cursor.fetchall()

```
total = 0
for fila in filas:
    total = total + fila[0]
media_python = total / len(filas)

cursor.execute("SELECT AVG(puntuacion) FROM peliculas")
media_sql = cursor.fetchone()[0]
Media calculada con Python: 7.74
Media calculada con SQL (AVG): 7.74
Mejor puntuada: Mar de silicio con 8.5
Drama → 2
Misterio → 1
Comedia → 1
Ciencia ficción → 1
Las dos medias coinciden (7.74), lo que confirma que el cálculo está
bien. No usé pandas porque todavía no lo hemos visto, pero me gustaría
probar a convertir estos resultados en un DataFrame.
```

Entregable: reflexión final

«Ahora entiendo que SQL y Python pueden trabajar juntos porque…»
…cada uno se encarga de una parte del trabajo. SQL es quien sabe pedirle a la base de datos justo los datos que necesito (filtrarlos, ordenarlos, contarlos) y Python es quien se conecta con sqlite3, ejecuta esa consulta, recibe los resultados y hace algo más con ellos: calcular una media, recorrerlos con un bucle o mostrarlos. La conexión funciona como un puente (connect, execute, fetchall y close), y si entiendo cada paso del puente, puedo cambiar la consulta SQL y obtener otra información sin tocar el resto del programa.